Figure 2.3. The type, amount, and target of work done by a team can be retrieved by querying your project-tracking and source-control management systems.

Figure 2.4. The component architecture for the system

Figure 2.5. Showing recidivism and bugs in a single graph because both should trend in the same direction

Figure 2.6. Breaking velocity out by effort after sorting data with tags

Figure 2.7. The chart graph showing how much code the team was changing over time. The Y-axis is in thousands to make it easier to read.

Figure 2.8. After another sprint recidivism has started creeping up and bugs are increasing sharply.

Figure 2.9. Velocity for both teams is starting to suffer.

Figure 2.10. The amount of code changed across both teams is skyrocketing.

Figure 2.11. A CodeFlower of a clean, well-written codebase

Figure 2.12. An excerpt from a CodeFlower generated from an extremely complex application

Figure 2.13. By adding data from the build and deployment system, the development team was able to add release dates to their analysis to track how changes they made to their process affected delivery time.

Figure 2.14. Adding releases to the dashboard. Now the team can see how their process affects the speed at which they release code.

Figure 2.15. The feedback loop as applied by our case-study team

Chapter 3. Trends and data from project-tracking systems

Figure 3.1. The first data source in your application lifecycle is your PTS. This is where tasks are defined and assigned, bugs are managed, and time is associated with tasks.

Figure 3.2. An example burn down chart

Figure 3.3. Velocity example